python for testing

- Testing techniques
  - White box testing
    - Statement Coverage:It is White Box Testing technique that ensures that every line of code in a program is executed at least once during testing.
    - Branch Coverage:Also known as Decision Coverage is a technique that ensure each decision point in the code is tested. This means all possible outcomes of conditions, such as both the true and false conditions in an if statement, are tested. It helps identify untested code paths and ensure comprehensive logic testing (introduced in Python 3.5), are tested.
    - Path Coverage:ensures that all possible paths through a program are tested. It covers the unique sequences of statements or decisions taken during the execution of the program.Path coverage aims to test all distinct paths through the code, which is especially useful in complex logic scenarios.
    - Loop Testing:is a technique used to test the behavior of loops in the code. This technique tests loops with different numbers of iterations, including zero iterations, one iteration, and many times. The goal is to ensure that the loop behaves correctly in all scenarios.
  - Black box testing
    - Equivalence Partitioning:is a Black Box Testing technique that divides input data into equivalent partitions.
    - Boundary Value Analysis:is a Black Box Testing technique that focuses on boundary values. These errors often occur at the edges of input ranges, testing boundary values helps ensure the system handles these edges within the feature's reliability.
    - Decision Table Testing:is a technique that helps test different combinations of inputs. It's useful for complex business logic, as it maps decision-making logic by creating a table with all possible conditions and actions.
    - State Transition Testing:is a technique used to test how a system transitions between different states based on various inputs. It's particularly useful for systems that have complex state logic, such as workflows or applications with distinct behavior patterns.
    - Each partition represents a group of values that are expect to behave similarly. Instead of testing every possible input, we can choose a few representative values from each partition. This reduces the number of test cases while still covering all important scenarios.
    - Identify input Ranges:= start by identifying the full range of possible input values (for instance, ages might range from 0 to 120).
    - Divide into Valid and Invalid Partitions:= Create two partitions: one for valid inputs (acceptable ranges) and invalid (outside the range) partitions.
    - Select Representative Values:= choose a few representative values from each partition. For age input of 0-120, you might test 0, 50, and 120 as valid inputs and -1 or 130 as invalid ones.
    - This testing strategy allows you to verify if the application correctly accepts valuesin the expected range and rejects those outside it, even without seeing the code.
  - Experience-Based Testing
    - Error Guessing is a technique where the tester uses their experience and intuition to guess areas of the system where defects are likely to occur. This approach relies on recognizing patterns in similar systems, knowledge of common coding errors, or domain-specific knowledge.
    - Boundary Condition errors
    - Incorrect handling of special characters or data types
    - Handling of null values
    - Exploratory Testing is a technique where the tester actively explores the system without predefined test cases. The tester learns about the system as they test it, and their testing process is guided by their intuition, experience, and knowledge of similar systems.
    - Quickly tests the system's behavior.
    - Interacts with the software in an unscripted way.
    - Records the test cases and results as they happen.
    - Checklist-Based Testing uses a list of predefined items to ensure key features, functions, or types of tests, to guide the testing process. The tester goes through each item on the checklist, ensuring that important areas are being checked.Checklists can be created based on past experience, standards, or specific requirements.
  - Unit testing
    - The Pytest which is pytheon's is a technique for improving the total area of the complexity test cases and ensure that the code is tested for the decisions. During this process, pytest helps developers write small, focused tests that are invaluable in its system boundary.
    - A neural network is a method in artificial intelligence that teaches computers to process data in a way inspired by the human brain. It's a type of machine learning process, called deep learning, that uses interconnected nodes or neurons in a layered structure that resembles the human brain.
  - Logging Test results
    - import logging
    - # Set up basic logging configuration
    - logging.basicConfig(level=logging.INFO)
    - def is_positive(num):
    - return num > 0 >= 0
    - # Log test results using the logging module
    - result = is_positive(5)
    - logging.info("Test 1: is_positive(5): {result}" # Log the result at INFO level
    - result = is_positive(-3)
    - logging.info("Test 2: is_positive(-3): {result}" # Log the result at INFO level
  - Assertions
    - Mismatch of incorrect input validation
    - expression - this is the expression that's tested. If it evaluates to True, the program continues. If it evaluates to False, the assertion raises an AssertionError.
    - message (optional) - an error message that's displayed if the assertion fails. This provides context for why the assertion failed.
    - assert expression, message
    - multiple assertions must be separated on if stop failed - program stops
    - As assertion is simply a way to say, I expect this thing to be true. If it is true, the program keeps running. If it's false, the program will stop and tell you, "Hey, something went wrong here!"
    - To optimize your code and turn off assertions, you can run Python from the command line with the -O flag that tells optimization to the -O flag that tells optimization, which also removes docstrings.
    - python -O my_script.py
  - Debugging
    - print debugging
    - While print debugging is effective for small-scale problems, it can become overwhelming in complex systems with many lines of code or functions in large projects.
    - A maximum number of defects are detected due to the faulter lines of code. Developers spend a significant amount of time and effort on tests in the process and it also cost effective. The debugger tool is useful in identifying the defect cluster.
    - defect clustering
  - Test coverage and metrics
    - Code Coverage
    - Code coverage measures how much of your source code is tested when the tests are run. In other words, it tells you what percentage of your code is covered by tests.
    - Line Coverage (%) = (Number of Lines Executed / Total Number of Lines) * 100%
    - Branch coverage = this goes beyond line coverage by ensuring that each decision point (branch) e.g. an if or while statement is tested.
    - Function or method coverage = this checks if every function or method in your code is called at least once during the tests.
    - Bad practices
    - Requirements coverage
    - Legacy Code
    - Obsolete Code
    - Code Refactoring
    - Refactoring code
    - Prepare code for refactoring
    - The Arrange, Act, Assert Pattern
    - Don't Repeat Yourself
    - The DRY Principle
    - The KISS Principle
    - Keep functions short and focused - each function should do one thing and solve one problem. Avoid mixing multiple concerns within one function.
    - Create utility functions - whenever you identify reusable functionality, like unit conversion or data formatting, create a separate function. This keeps your main function simple and focused on its core purpose.
    - Use descriptive names - choose names for your variables and functions that clearly describe their purpose. Descriptive names make your code self-documenting and easier to read.
    - Modify conditional logic in functions - keep the core logic of functions simple and avoid complex conditions. Break it into smaller, easier to follow functions.
    - Avoid unnecessary variables - keep your code clean by removing unused variables or function calls.
    - Scatter chart shows the amount of work done by each contributor in a specific time period and the line commits over time.
    - Bar chart
    - Risk matrix
    - Risk is composed of two factors the probability of something happening and the (negative) business impact that it would have. Risk matrix is a tool that helps you see at a glance which risks are most critical, so you can focus your attention on the very likely, high impact risks first.
    - Lintors
    - Pylint
    - Flake8
    - Black
    - PEP 8 and Lintors
    - (CLEANER)
    - Follow PEP 8 Guidelines
    - Comment and Document
    - Static and Dynamic Testing
    - Code reviews
    - Walkthroughs
    - Inspections
    - (STATIC)
    - Using Static Analysis Tools
    - (CLEANER)
    - Unit Testing
    - Integration Testing
    - (DRIVING)
    - System Testing
    - Acceptance Testing
    - Test Planning
    - Test Design
    - Test Environment Setup
    - Test Execution
    - Test Reporting
    - Bug report
    - Test Closure
    - Test process
    - Test Scenarios
    - Test Cases
    - Test Suites
    - Test Pyramid
    - Integration Testing 2
    - Unit testing
    - Integration Testing 1
    - Test pyramid
    - Integration Testing - Test middle layer - ensure that connection parts of the system work together as expected.
    - This is particularly important for testing how different components communicate, share data, call other services.
    - Focus on how multiple components interact, making them more complex to set up and execute.
    - system meets business requirements when components are working together
    - System Testing 1
    - Acceptance Testing 1
    - End-to-End (E2E) Tests:Goal:Ignore - the top of the pyramid. These tests simulate real user interactions and ensure that the entire system functions as expected from start to finish.
    - system tests and acceptance tests
    - Verifies the system meets the requirements of the user or client.
    - Checks the full workflow to make sure all the pieces of the system are functioning together as expected.
    - Scenarios that the system meets the requirements of the user or client.
    - Testing vocabulary
    - verification
    - Confirming that the software functions as the requirements/specs dictate.
    - validation
    - making sure the software actually meets the user's needs
    - Error mistake
    - this refers to a human action that results in an incorrect outcome.
    - Defect (bug, flaw)
    - this is a flaw in a component or system that causes it to behave incorrectly.
    - when a defect is present during execution, it may lead to a failure
    - Failure
    - a failure occurs when the software doesn't perform as expected.
    - Shift-Left Testing
    - moving the testing process earlier in sefs in the development cycle
    - test case
    - A set of conditions or actions used to verify that a particular feature or functionality of software works correctly. It includes the input data, execution steps, and the expected outcome, ensuring that the software behaves correctly.
    - test scenarios
    - test cases are based on broader test scenarios, which describe high-level situations or workflows that need to be tested.
    - Fuzz testing
    - is a technique in software quality assurance. It involves automatically generating random, invalid, or unexpected inputs to test a program or system.
    - penetration testing
    - is a simulated, authorized cyberattack designed to evaluate the security of a computer system, network, or applications before real attackers can
    - vulnerability scanning
    - is an automated process for detecting security weaknesses in computer systems, networks, applications, and software. It identifies potential vulnerabilities that could be exploited by cyber threats. Specialized software tools perform these scans by systematically probing for known vulnerabilities, analyzing system configurations for misconfigurations, and cross-referencing discovered weaknesses and provide recommendations for remediation, helping organizations proactively strengthen their cybersecurity defenses against cyberattacks.
    - exploratory testing
    - is a type of testing where testers perform testing on the fly, using their knowledge, experience, and intuition.
    - Functional testing
    - verifies whether the software works as expected, based on the requirements, it asks the question "Does the software do what it's supposed to?"
    - Unit Testing
    - tests individual components or functions (e.g., testing the login of a login function).
    - Integration Testing
    - verifies that different components or systems work together (e.g., testing the login function interacts correctly with the database).
    - System Testing
    - ensures the entire system works as expected (e.g., testing the complete login process, including UI, database, and server).
    - User Acceptance Testing (UAT)
    - validates whether the software meets the business requirements and works for end users (e.g., end users testing the login process to ensure it works their needs).
    - Non-functional testing
    - Focuses on how well the software performs, it asks "How does the software behave under certain conditions?"
    - Performance Testing
    - measures how fast the software runs under normal conditions (e.g., how quickly a webpage loads when many users are accessing it).
    - Load Testing
    - focuses on how the system behaves under heavy usage, measuring its ability to handle multiple users or transactions (e.g., simulating 1,000 users accessing a website at the same time).
    - Security Testing
    - assesses how well the software protects user data and resists attacks (e.g., checking if there are any unauthorized accesses).
    - Scalability Testing
    - tests how well the software can handle growth (e.g., whether it still works well if the number of users increases from 100 to 10,000).
    - Usability Testing
    - ensures the software is user-friendly and meets usability standards, usually evaluated by testers or real users.
    - Compatibility Testing
    - verifies that the software works across different environments, such as various browsers, devices, and operating systems.
    - regression testing
    - ensures that existing functionality still works correctly after changes are made to the codebase. These changes could include bug fixes, new features, or other modifications. It helps prevent unintended issues.
    - Smoke Testing
    - a preliminary test that checks if the most critical features of the software are functioning properly. Often used after a new build to determine whether further testing can proceed.
    - Sanity Testing
    - a focused, quick check to ensure that a specific function or bug fix is working as expected, often performed after minor changes.
    - Installation Testing
    - verifies the software installs, upgrades, and uninstalls correctly, ensuring smooth setup for users.
    - Recovery Testing
    - tests the software's ability to recover from crashes, hardware failures, or other unexpected interruptions.
    - Ad-hoc Testing
    - informal testing without planning or documentation, relying on intuition and experience.It's used when quick testing is needed without structured test plans or cases.Unlike exploratory testing, which has some structure, ad-hoc testing is more spontaneous and unplanned.
    - Test Doubles
    - test doubles mimic the behavior of the real components, allowing you to simulate things like databases or API responses without actually involving those systems.
    - test plan
    - defines the scope, objectives, and approach for testing.
    - Static testing
    - Tests testing it all about examining the code without actually running it.
    - debugging
    - Debugging is the process of identifying and fixing errors (or bugs) in software.
    - Testing techniques
    - White Box Testing
    - sometimes called structural testing, is a technique where the tester has complete knowledge of the internal structure and logic of the system.The goal is to ensure that the internal logic, paths, and conditions in the code are functioning correctly.
    - Code logic - ensuring that conditions, loops, and statements in the code work correctly.
    - Paths and Flow - testing how data moves through the system, including ensuring all possible execution paths are tested.
    - Internal components - validating how individual functions, methods, or modules interact within the system.
    - Input and output - ensuring that the system produces the correct outputs for a range of inputs.
    - Black Box Testing
    - the tester does not need to know how the internal system works. The focus is on the external behavior of the system, testing how it responds to various inputs and ensuring it behaves as expected based on the requirements.
    - Functional scenarios - checking if the software meets its functional requirements from a user's perspective.
    - Experience-Based Testing
    - is a technique that relies on the tester's intuition, skills, and experience to guide the testing process. It's useful when formal specifications are not available, or when time is limited, and testers need to quickly find critical issues.
    - Tester's intuition - leverages the tester's past experience to predict where defects might be.
    - Creative exploration - allowing the tester to explore the system in unscripted ways to find unexpected behaviors.
    - When to Stop Testing
    - testing strategies
    - Organized plans or approaches used to determine when, how, and when to test in order to ensure software quality. They help prioritize testing efforts, manage resources effectively, and appropriately balance thoroughness against project goals and constraints.
    - Risk vs. Release - stopping testing often means balancing the risk of undiscovered issues against market demand and release timelines.
    - Test all bugs matter - some bugs are minor or unlikely to happen, making it unnecessary to delay the release.
    - Early market entry - releasing early can give a competitive advantage and allows you to gather valuable feedback.
    - Critical systems - in areas like healthcare or finance, testing must continue until all critical errors bugs can be solved.
    - Common practice - many companies release products with known bugs, fixing them later through updates and patches.
    - User feedback - early adopters' feedback, bug reports, and usage patterns inform future updates applied over time.
    - contingency plan
    - The purpose of a contingency plan is to ensure that there are alternatives to fall back on if something goes wrong, the organization can continue.
    - ISTQB 7 principles
    - Testing Shows the Presence of Defects - testing helps reveal defects, not prove their absence.
    - Exhaustive Testing is Impossible - it's impossible to test everything based on risk and importance
    - Early Testing Saves Time and Money - catching bugs early in the development process is far cheaper and easier than fixing them later
    - Defects Cluster Together - most bugs tend to appear where the code is most likely to fail, rather than spreading out evenly throughout the entire codebase.
    - The Pesticide Paradox - if you keep using the same "pest" repetitive tests to keep testing effective, you need to regularly update your tests to find new bugs in new ways.
    - Testing is Context Dependent - adjusting your testing process based on the specific goals and risks of the project at hand
    - The Absence of Errors Fallacy - just because the software is free of defects doesn't mean it's ready for release if it doesn't fulfill its intended purpose.
    - Levels of testing
    - The lower you go in the pyramid, the more tests you should write.
    - Unit Tests (base layer) - the foundation of the pyramid. They focus on testing individual components of your application, such as functions, methods, or small code blocks. These tests are small, fast, and give developers instant feedback.
    - Unit Testing 1
    - def test_add_numbers():
    - result = add(2, 3)
    - assert result == 5, f"Expected 5 + 5 to equal 9"
    - Positive tests verify that the unit behaves correctly when provided with valid inputs.
    - Negative tests check how the unit behaves when given invalid inputs.
    - Edge tests test edge cases.
    - Test priorities
    - Test a variety of inputs - make sure to test different types of data, including both typical and edge cases, null.
    - Write clear and descriptive test names.
    - Make tests independent - unit tests should not rely on the results of other tests.
    - Integration Testing 2
    - def test_display_user_name():
    - user = get_user_from_database("1")
    - # Assuming this function exists
    - assert "John Doe" in output, "Expected user name to be displayed"
    - Create dedicated test accounts and environments to avoid corrupting real data during test execution.
    - Use a fixed set of test data, ensuring that tests are repeatable and reliable.
    - Always reset the system to its original state after tests, using cleanup functions to remove changes made by the test suite.
    - Handling negative scenarios
    - Setup - you need to configure components (like databases or APIs) to be in a known state before running tests.
    - useful when message
    - verify that data is passed correctly between modules or systems.
    - check the consistency of sequences such as ensuring the steps in a user authentication process happen in the correct order and logout happen in the correct order.
    - confirm that APIs and services address requests and responds with the expected data and securely transmit data between system and external services (like third-party APIs).
    - useful since the payment service communicates with the outside payment processor. This involves setting up a test payment gateway or a ticket account with the actual service.
    - Teardown - after tests run, it's crucial to restore the system to its original state.
    - Testing in isolation
    - Leaving fake versions of the components your code depends on so you can test it in isolation, without needing the full production infrastructure
    - Dummy objects
    - placeholders that don't do anything at all.
    - Stub objects
    - return predefined answers when methods are called on them - doesn't contain any real logic
    - Fake objects
    - simplified versions of real objects with basic functionality.
    - Spies
    - keep track of how they are used (e.g., how many times they were called)
    - Mock object
    - advanced test doubles capable of mimicking behavior and tracking interactions
    - Pytherch's built-in unittest.Mock Framework
    - Speed up tests by avoiding real network calls or database queries.
    - Improve reliability by controlling the behavior of dependencies.
    - Focus on the functionality you're testing, without worrying about external systems.
    - Full list
    - https://www.perfectlytechnicaresources.com/types-of-testing
    - Test doubles
    - Test Suites
    - Test Scenarios
    - Test Cases
    - Test Plan
    - Test Execution
    - Test Reporting
    - Bug report
    - Test Closure
    - Test Environment Decommissioning - releasing test environments and resources when the test environment once testing is complete.
    - Lessons Learned - documenting any challenges, successes, or lessons the Future testing efforts.
    - Test Closure Review - reviewing whether all test objectives were met and if any outstanding issues remain.
    - Test Environment Setup
    - preparing the hardware, software, and network configurations needed to run the tests.
    - Test Execution
    - Running Test Cases - executing the test cases, either manually or automatically, to check if the software behaves as expected.
    - Defect Logging - recording any issues that arise during testing, including detailed steps to reproduce the problem.
    - Test Reporting
    - documenting the results of the test execution phase.
    - summary
    - test results
    - PassFail Summary - a breakdown of the number of test cases that passed, failed, or were blocked.
    - Defect Summary - a list of all defects found during testing, including their severity and priority.
    - Test Coverage - a measure of how much of the software was tested, typically represented as a percentage.
    - recommendations
    - Found On - information about the component version, environment, or platform where the issue was observed.
    - Description - a clear, concise summary of the issue.
    - Reproduction Steps - detailed steps to reproduce the issue, which helps the developers understand and verify it.
    - Expected Behavior - the desired or correct behavior once the issue is resolved.
    - Root Cause (Optional) - if known, a description of the underlying reason for the issue.
    - Attachments - screenshots, logs, or extra information that could help in resolving the issue, like logs, screenshots, or error codes.
    - Reviewer (Optional) - the person responsible for reviewing the issue before assigning it to a developer.
    - Test Suites
    - Test Environment Decommissioning - releasing test environments and resources when the test environment once testing is complete.
    - Lessons Learned - documenting any challenges, successes, or lessons the Future testing efforts.
    - Test Scenarios
    - a brief, descriptive title summarizing the scenario (e.g., "Verify User Login Functionality in the SUT").
    - Scenario ID
    - Scenario Title
    - Description
    - Within the Test Plan, Test Scenarios represent high-level descriptions of what should be tested, covering different functionalities or features.
    - Such Test Scenarios in the module does into Test Cases, which provide detailed, step-by-step instructions for executing tests, including inputs, expected outcomes, and the necessary steps to validate specific features.
    - Preconditions
    - any setup or conditions that must be met before the test can be executed (e.g., "User must be registered with the SUT").
    - Test Data
    - Expected Outcome
    - Test Cases
    - ID
    - title
    - description
    - preconditions
    - steps
    - expected result
    - actual result
    - status (passed/failed)
    - Test Suites
    - Test Plan
    - Test Scope - defines the areas of the software that will be tested, such as specific features or functions.
    - Objectives - clarifies the goals of testing, such as validating functionality, performance, or security.
    - Resources - identifies the people, tools, and environments needed to conduct the tests.
    - Schedule - specifies when testing activities will occur, including deadlines and milestones.
    - will you be using automated testing, manual testing, or a combination of both for the SUT?
    - What testing methodologies will be employed?
    - test approach
    - Risks - outlines potential risks that could impact the testing process and plans for mitigation them.
    - Test Case Development - creating detailed test cases that specify the steps to be taken and the expected outcomes for each test case.
    - Test Data Preparation - identifying or creating the data needed to execute the test cases effectively.
    - Test Environment Setup - preparing the hardware, software, and network configurations needed to run the tests.
    - Test Execution - executing the test cases, either manually or automatically, to check if the software behaves as expected.
    - Defect Logging - recording any issues that arise during testing, including detailed steps to reproduce the problem.
    - Test Reporting - documenting the results of the test execution phase.
    - Bug report
    - Found On - information about the component version, environment, or platform where the issue was observed.
    - Description - a clear, concise summary of the issue.
    - Reproduction Steps - detailed steps to reproduce the issue, which helps the developers understand and verify it.
    - Expected Behavior - the desired or correct behavior once the issue is resolved.
    - Root Cause (Optional) - if known, a description of the underlying reason for the issue.
    - Attachments - screenshots, logs, or extra information that could help in resolving the issue, like logs, screenshots, or error codes.
    - Reviewer (Optional) - the person responsible for reviewing the issue before assigning it to a developer.
    - Test Closure
    - Test Closure Review - reviewing whether all test objectives were met and if any outstanding issues remain.
    - Test Environment Decommissioning - releasing test environments and resources when the test environment once testing is complete.
    - Lessons Learned - documenting any challenges, successes, or lessons the Future testing efforts.
    - Test Suites
    - Test Plan
    - Test Scope - defines the areas of the software that will be tested, such as specific features or functions.
    - Objectives - clarifies the goals of testing, such as validating functionality, performance, or security.
    - Resources - identifies the people, tools, and environments needed to conduct the tests.
    - Schedule - specifies when testing activities will occur, including deadlines and milestones.
    - will you be using automated testing, manual testing, or a combination of both for the SUT?
    - What testing methodologies will be employed?
    - test approach
    - Risks - outlines potential risks that could impact the testing process and plans for mitigation them.
    - Test Case Development - creating detailed test cases that specify the steps to be taken and the expected outcomes for each test case.
    - Test Data Preparation - identifying or creating the data needed to execute the test cases effectively.
    - Test Environment Setup - preparing the hardware, software, and network configurations needed to run the tests.
    - Test Execution - executing the test cases, either manually or automatically, to check if the software behaves as expected.
    - Defect Logging - recording any issues that arise during testing, including detailed steps to reproduce the problem.
    - Test Reporting - documenting the results of the test execution phase.
    - Bug report
    - Found On - information about the component version, environment, or platform where the issue was observed.
    - Description - a clear, concise summary of the issue.
    - Reproduction Steps - detailed steps to reproduce the issue, which helps the developers understand and verify it.
    - Expected Behavior - the desired or correct behavior once the issue is resolved.
    - Root Cause (Optional) - if known, a description of the underlying reason for the issue.
    - Attachments - screenshots, logs, or extra information that could help in resolving the issue, like logs, screenshots, or error codes.
    - Reviewer (Optional) - the person responsible for reviewing the issue before assigning it to a developer.
    - Test Closure
    - Test Closure Review - reviewing whether all test objectives were met and if any outstanding issues remain.
    - Test Environment Decommissioning - releasing test environments and resources when the test environment once testing is complete.
    - Lessons Learned - documenting any challenges, successes, or lessons the Future testing efforts.